

Exercices MySQL - Gestion d'un Système de Réservation Hôtelière

Partie 1 : Exercices DDL (Data Definition Language)

Contexte

Vous êtes chargé de créer une base de données pour un système de réservation hôtelière. Cette base de données doit gérer les hôtels, les chambres, les clients et les réservations.

Exercice 1.1 : Création de la base de données

Créez une base de données nommée "hotel_reservation" et utilisez-la pour les exercices suivants.

```
-- Voici la solution
CREATE DATABASE hotel_reservation;
USE hotel_reservation;
```

Exercice 1.2 : Création des tables

Pour chaque table, vous devez créer les attributs demandés et appliquer les contraintes nécessaires. Voici les détails pour chaque table :

Table des hôtels (hotels):

- Identifiant unique auto-incrémenté (hotel_id)
- Nom de l'hôtel (100 caractères maximum, obligatoire)
- Adresse (255 caractères maximum, obligatoire)
- Ville (50 caractères maximum, obligatoire)
- Pays (50 caractères maximum, obligatoire)
- Nombre d'étoiles (entier)
- Date de création (obligatoire)

Table des types de chambres (types_chambres):

- Identifiant unique auto-incrémenté (type_id)
- Libellé du type (50 caractères maximum, obligatoire)
- Description (texte)
- Capacité en nombre de personnes (obligatoire)

Table des chambres (chambres):

- Identifiant unique auto-incrémenté (chambre_id)
- Référence à l'hôtel (obligatoire, lié à la table hôtels)
- Numéro de chambre (10 caractères maximum, obligatoire)
- Référence au type de chambre (obligatoire, lié à la table types_chambres)
- Prix par nuit (nombre décimal, obligatoire)
- Étage (entier)
- Disponibilité (booléen, valeur par défaut : true)

Table des clients (clients):

- Identifiant unique auto-incrémenté (client_id)
- Nom (50 caractères maximum, obligatoire)
- Prénom (50 caractères maximum, obligatoire)
- Email (100 caractères maximum, obligatoire et unique)
- Téléphone (20 caractères maximum)
- Date de naissance
- Adresse (255 caractères maximum)
- Ville (50 caractères maximum)
- Pays (50 caractères maximum)
- Date d'inscription (date et heure, valeur par défaut : date/heure actuelle)

Table des réservations (reservations):

- Identifiant unique auto-incrémenté (reservation_id)
- Référence au client (obligatoire, lié à la table clients)
- Référence à la chambre (obligatoire, lié à la table chambres)
- Date de début (obligatoire)
- Date de fin (obligatoire)
- Nombre de personnes (obligatoire)
- Prix total (nombre décimal, obligatoire)
- Statut (chaîne de caractères, valeur par défaut : 'en attente')
- Date de réservation (date et heure, valeur par défaut : date/heure actuelle)

-- Voici la solution pour la table des hôtels

```
CREATE TABLE hotels (  
    hotel_id INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(100) NOT NULL,  
    adresse VARCHAR(255) NOT NULL,  
    ville VARCHAR(50) NOT NULL,  
    pays VARCHAR(50) NOT NULL,  
    nombre_etoiles INT,  
    date_creation DATE NOT NULL  
);
```

-- Table des types de chambres

```
CREATE TABLE types_chambres (  
    type_id INT AUTO_INCREMENT PRIMARY KEY,  
    libelle VARCHAR(50) NOT NULL,  
    description TEXT,  
    capacite INT NOT NULL  
);
```

-- Table des chambres

```
CREATE TABLE chambres (  
    chambre_id INT AUTO_INCREMENT PRIMARY KEY,  
    hotel_id INT NOT NULL,  
    numero VARCHAR(10) NOT NULL,  
    type_id INT NOT NULL,  
    prix_nuit DECIMAL(10, 2) NOT NULL,  
    etage INT,  
    disponible BOOLEAN DEFAULT TRUE,  
    -- Clés étrangères  
    FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id) ON DELETE CASCADE,  
    FOREIGN KEY (type_id) REFERENCES types_chambres(type_id)  
);
```

-- Table des clients

```
CREATE TABLE clients (  
    client_id INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,
```

```

    prenom VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    telephone VARCHAR(20),
    date_naissance DATE,
    adresse VARCHAR(255),
    ville VARCHAR(50),
    pays VARCHAR(50),
    date_inscription TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table des réservations
CREATE TABLE reservations (
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,
    client_id INT NOT NULL,
    chambre_id INT NOT NULL,
    date_debut DATE NOT NULL,
    date_fin DATE NOT NULL,
    nombre_personnes INT NOT NULL,
    prix_total DECIMAL(10, 2) NOT NULL,
    statut VARCHAR(20) DEFAULT 'en attente',
    date_reservation TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    -- Clés étrangères
    FOREIGN KEY (client_id) REFERENCES clients(client_id),
    FOREIGN KEY (chambre_id) REFERENCES chambres(chambre_id)
);

```

Exercice 1.3 : Modifications de structure (ALTER TABLE)

Effectuez les modifications suivantes sur les tables existantes :

1. Ajoutez une colonne "responsable" (chaîne de 100 caractères) à la table des hôtels.
2. Modifiez le type de la colonne "téléphone" des clients pour accepter 30 caractères au lieu de 20.
3. Ajoutez une colonne "commentaire" (texte) à la table des réservations.

```

-- Solution pour les modifications de tables
-- 1. Ajouter une colonne pour le responsable d'hôtel
ALTER TABLE hotels ADD COLUMN responsable VARCHAR(100);

-- 2. Modifier le type de la colonne téléphone des clients
ALTER TABLE clients MODIFY COLUMN telephone VARCHAR(30);

-- 3. Ajouter une colonne commentaire à la table des réservations
ALTER TABLE reservations ADD COLUMN commentaire TEXT;

```

Partie 2 : Script d'insertion de données (DML - Data Manipulation Language)

Écrivez des instructions INSERT pour ajouter les données suivantes :

Exercice 2.1 : Insertion des hôtels

- Grand Hôtel (Paris, France, 5 étoiles)
- Hôtel du Port (Marseille, France, 4 étoiles)
- Mountain View (Chamonix, France, 3 étoiles)

Exercice 2.2 : Insertion des types de chambres

- Standard (capacité 2 personnes)
- Deluxe (capacité 2 personnes)
- Suite (capacité 4 personnes)
- Familiale (capacité 5 personnes)

Exercice 2.3 : Insertion des chambres

- 3 chambres dans le Grand Hôtel (différents types et prix)
- 2 chambres dans l'Hôtel du Port
- 2 chambres dans le Mountain View

Exercice 2.4 : Insertion des clients

- Au moins 3 clients avec des informations complètes

Exercice 2.5 : Insertion des réservations

- Au moins 2 réservations pour différents clients et hôtels

-- Insertion des hôtels

```
INSERT INTO hotels (nom, adresse, ville, pays, nombre_etoiles, date_creation) VALUES
('Grand Hôtel', '15 rue de la Paix', 'Paris', 'France', 5, '2000-06-15'),
('Hôtel du Port', '25 avenue de la Mer', 'Marseille', 'France', 4, '2005-03-22'),
('Mountain View', '8 rue des Alpes', 'Chamonix', 'France', 3, '2010-12-10');
```

-- Insertion des types de chambres

```
INSERT INTO types_chambres (libelle, description, capacite) VALUES
('Standard', 'Chambre simple avec les commodités de base', 2),
('Deluxe', 'Chambre spacieuse avec vue', 2),
('Suite', 'Grande suite avec salon séparé', 4),
('Familiale', 'Chambre adaptée pour familles', 5);
```

-- Insertion des chambres

```
INSERT INTO chambres (hotel_id, numero, type_id, prix_nuit, etage, disponible) VALUES
(1, '101', 1, 120.00, 1, TRUE),
(1, '201', 2, 180.00, 2, TRUE),
(1, '301', 3, 300.00, 3, TRUE),
(2, '101', 1, 100.00, 1, TRUE),
(2, '201', 4, 220.00, 2, TRUE),
(3, '101', 1, 90.00, 1, TRUE),
(3, '201', 2, 150.00, 2, TRUE);
```

-- Insertion des clients

```
INSERT INTO clients (nom, prenom, email, telephone, date_naissance, adresse, ville, pays)
VALUES
('Dupont', 'Jean', 'jean.dupont@email.com', '+33 6 12 34 56 78', '1980-05-15', '10 rue Victor
Hugo', 'Paris', 'France'),
('Martin', 'Sophie', 'sophie.martin@email.com', '+33 6 23 45 67 89', '1985-09-22', '5 avenue
Foch', 'Lyon', 'France'),
('Bernard', 'Michel', 'michel.bernard@email.com', '+33 7 34 56 78 90', '1975-11-30', '18
boulevard Gambetta', 'Marseille', 'France');
```

-- Insertion des réservations

```
INSERT INTO reservations (client_id, chambre_id, date_debut, date_fin, nombre_personnes,
prix_total, statut) VALUES
(1, 3, '2023-07-10', '2023-07-15', 2, 1500.00, 'confirmée'),
(2, 5, '2023-08-01', '2023-08-05', 4, 880.00, 'confirmée');
```

Partie 3 : Exercices DML (Data Manipulation Language)

Exercice 3.1 : Requêtes SELECT simples

Réalisez les requêtes SELECT suivantes :

1. Afficher la liste de tous les hôtels avec leur nom et leur nombre d'étoiles.
2. Lister toutes les chambres de l'hôtel "Grand Hôtel" avec leur numéro et prix.
3. Afficher les clients qui habitent à Paris.

-- Solution pour l'exercice 3.1.1

```
SELECT nom, nombre_etoiles
FROM hotels;
```

-- Solution pour l'exercice 3.1.2

```
SELECT c.numero, c.prix_nuit
FROM chambres c
JOIN hotels h ON c.hotel_id = h.hotel_id
WHERE h.nom = 'Grand Hôtel';
```

-- Solution pour l'exercice 3.1.3

```
SELECT nom, prenom
FROM clients
WHERE ville = 'Paris';
```

Exercice 3.2 : Requêtes avec JOIN

Réalisez les requêtes suivantes en utilisant des jointures :

1. Afficher les détails des réservations avec le nom du client, le nom de l'hôtel et le numéro de chambre.
2. Lister les chambres qui n'ont jamais été réservées.

-- Solution pour l'exercice 3.2.1

```
SELECT r.reservation_id, c.nom, c.prenom, h.nom AS hotel, ch.numero
FROM reservations r
JOIN clients c ON r.client_id = c.client_id
JOIN chambres ch ON r.chambre_id = ch.chambre_id
JOIN hotels h ON ch.hotel_id = h.hotel_id;
```

-- Solution pour l'exercice 3.2.2

```
SELECT h.nom AS hotel, ch.numero
FROM chambres ch
JOIN hotels h ON ch.hotel_id = h.hotel_id
LEFT JOIN reservations r ON ch.chambre_id = r.chambre_id
WHERE r.reservation_id IS NULL;
```

Exercice 3.3 : Requêtes avec GROUP BY et fonctions d'agrégation

Réalisez les requêtes suivantes en utilisant des groupements et des fonctions d'agrégation :

1. Calculer le prix moyen des chambres par type de chambre.
2. Calculer le montant total des réservations par client.

-- Solution pour l'exercice 3.3.1

```
SELECT t.libelle, AVG(c.prix_nuit) AS prix_moyen
FROM chambres c
JOIN types_chambres t ON c.type_id = t.type_id
GROUP BY t.libelle;
```

-- Solution pour l'exercice 3.3.2

```
SELECT c.nom, c.prenom, SUM(r.prix_total) AS montant_total
FROM reservations r
JOIN clients c ON r.client_id = c.client_id
GROUP BY c.client_id, c.nom, c.prenom;
```

Exercice 3.4 : Requêtes UPDATE et DELETE

Réalisez les modifications suivantes sur les données :

1. Augmenter de 10% le prix des chambres dans les hôtels 5 étoiles.
2. Modifier l'adresse email du client "Dupont" pour « jean.dupont@nouveauemail.com ».

-- Solution pour l'exercice 3.4.1

```
UPDATE chambres c
JOIN hotels h ON c.hotel_id = h.hotel_id
SET c.prix_nuit = c.prix_nuit * 1.1
WHERE h.nombre_etoiles = 5;
```

-- Solution pour l'exercice 3.4.2

```
UPDATE clients
SET email = 'jean.dupont@nouveauemail.com'
WHERE nom = 'Dupont' AND prenom = 'Jean';
```

Partie 4 : Exercices TCL (Transaction Control Language)

Exercice 4.1 : Transactions simples

Une transaction est un ensemble d'opérations qui doivent être exécutées comme un tout, de manière atomique. Si une partie échoue, toutes les opérations sont annulées.

Consigne: Créez une transaction qui effectue les opérations suivantes :

1. Insérer une nouvelle réservation
2. Mettre à jour la disponibilité de la chambre concernée

-- Solution pour l'exercice 4.1

```
START TRANSACTION;
```

-- Insérer une nouvelle réservation

```
INSERT INTO reservations (client_id, chambre_id, date_debut, date_fin, nombre_personnes,
prix_total, statut)
VALUES (3, 8, '2023-11-15', '2023-11-20', 1, 375.00, 'confirmée');
```

-- Mettre à jour le statut de disponibilité de la chambre

```
UPDATE chambres SET disponible = FALSE WHERE chambre_id = 8;
```

```
COMMIT;
```

Exercice 4.2 : Transactions avec ROLLBACK

Le ROLLBACK permet d'annuler toutes les opérations effectuées depuis le début d'une transaction.

Consigne: Écrivez une transaction qui teste d'abord une mise à jour puis l'annule avec ROLLBACK.

```
-- Solution pour l'exercice 4.2
START TRANSACTION;

-- Augmenter temporairement tous les prix de 50%
UPDATE chambres SET prix_nuit = prix_nuit * 1.5;

-- Afficher les nouveaux prix (pour vérification)
SELECT numero, prix_nuit FROM chambres LIMIT 5;

-- Annuler la modification
ROLLBACK;

-- Vérifier que les prix sont revenus à leur valeur initiale
SELECT numero, prix_nuit FROM chambres LIMIT 5;
```

Partie 5 : Exercices DCL (Data Control Language)

Le DCL (Data Control Language) permet de gérer les permissions d'accès à la base de données.

Exercice 5.1 : Création d'utilisateurs

Le DCL permet de créer des utilisateurs pour gérer les accès à la base de données.

Consigne: Créez les utilisateurs suivants :

1. Un utilisateur administrateur (hotel_admin)
2. Un utilisateur pour la réception (reception)

```
-- Solution pour l'exercice 5.1
-- Création d'utilisateurs
CREATE USER 'hotel_admin'@'localhost' IDENTIFIED BY 'admin_pass';
CREATE USER 'reception'@'localhost' IDENTIFIED BY 'reception_pass';
```

Exercice 5.2 : Attribution de privilèges

Le DCL permet d'attribuer différents niveaux d'accès aux utilisateurs.

Consigne: Attribuez les privilèges suivants :

1. Pour l'utilisateur hotel_admin : tous les privilèges sur toutes les tables
2. Pour l'utilisateur reception :
 - Lecture sur toutes les tables
 - Création et modification de réservations
 - Création et modification de clients
 - Modification de la disponibilité des chambres

```
-- Solution pour l'exercice 5.2
-- Attribution des privilèges à l'administrateur
GRANT ALL PRIVILEGES ON hotel_reservation.* TO 'hotel_admin'@'localhost';
```

```
-- Attribution des privilèges à la réception
GRANT SELECT ON hotel_reservation.* TO 'reception'@'localhost';
GRANT INSERT, UPDATE ON hotel_reservation.reservations TO 'reception'@'localhost';
GRANT INSERT, UPDATE ON hotel_reservation.clients TO 'reception'@'localhost';
GRANT UPDATE ON hotel_reservation.chambres TO 'reception'@'localhost';
```

Exercice 5.3 : Révocation de privilèges

Parfois, il est nécessaire de révoquer des privilèges précédemment accordés.

Consigne: Modifiez les privilèges suivants :

1. Révoquez le droit de suppression (DELETE) pour l'utilisateur réception sur toutes les tables

```
-- Solution pour l'exercice 5.3
-- Révoquer certains privilèges à l'utilisateur réception
REVOKE DELETE ON hotel_reservation.* FROM 'reception'@'localhost';
```